Ruta de estudio: componentes FXML, estilos CSS y eventos en JavaFX

Programación Orientada a Objetos · DUOC UC · JavaFX 21 · JDK 21 · Maven 3.9

> Esta ruta es para que estudies **por tu cuenta** antes de la próxima semana. Cada etapa tiene teoría corta, **código listo para copiar**, cosas para **probar y modificar**, preguntas de comprensión y un reto. La idea es aprender **haciendo**: copia, ejecuta, cambia algo y observa qué pasa.

Índice

1. Cómo usar esta ruta
2. Etapa 0 · Preparar tu laboratorio
3. Etapa 1 · Componentes de UI en FXML
4. Etapa 2 · Eventos de usuario
5. Etapa 3 · Estilos CSS en JavaFX
6. Etapa 4 · Reto integrador: formulario de registro
7. Anexo A · Resumen de componentes
8. Anexo B · Resumen de eventos
9. Anexo C · Resumen de CSS
10. Anexo D · Errores frecuentes
11. Anexo E · Glosario y recursos
12. Checklist final

---

1. Cómo usar esta ruta

Qué vas a lograr

- Reconocer y usar los **componentes** más importantes de JavaFX (layouts, controles, menús, tablas) desde **FXML** y desde Scene Builder.
- Reaccionar a lo que hace el usuario: **eventos** de botón, mouse, teclado y cambios de valor.
- **Dar estilo** a una aplicación con **CSS**: clases, ids, estados, variables y un tema oscuro.
- Unir todo en un formulario de registro completo.

Plan sugerido (5 sesiones de ~1 hora)

Sesión	Etapa	Qué haces	Al terminar puedes…
1	**0 + 1A**	Preparas el laboratorio. Pruebas layouts.	Armar una pantalla con `BorderPane`, `VBox`, `HBox` y `GridPane`
2	**1B + 1C**	Controles de formulario, menús, pestañas, tablas	Elegir el componente correcto para cada dato
3	**2**	Eventos de clic, mouse, teclado y valores	Hacer que la pantalla responda al usuario
4	**3**	CSS: clases, ids, estados, variables y tema oscuro	Cambiar la apariencia sin tocar el código
5	**4**	Reto integrador	Construir un formulario completo por tu cuenta

Reglas para estudiar bien

1. **No solo copies:** después de cada ejemplo haz los ejercicios de la sección **“Pruébalo”**.
2. **Rompe cosas a propósito.** Borra un `fx:id` y mira el error; es la mejor forma de aprender a leer errores.
3. **Anota lo que no entiendas** y llévalo al foro con el formato: *qué intentabas · qué hiciste · qué pasó · qué probaste*.
4. Abre los `.fxml` también en **Scene Builder** (clic derecho → *Open in SceneBuilder*) y mira cómo se ve cada cosa.
5. Haz primero el reto por tu cuenta y **después** compara con tus compañeros en el foro.

---

2. Etapa 0 · Preparar tu laboratorio

Requisitos

✓	Herramienta	Cómo comprobarlo
☐	JDK 21	`java -version`
☐	Maven 3.9+ (o el de IntelliJ)	`mvn -v`
☐	IntelliJ IDEA	Abre y crea un proyecto
☐	Scene Builder	Ver la guía *JavaFX con FXML*, sección 4
☐	Haber leído las guías **JavaFX base** y **JavaFX con FXML**	—

Paso 1 · Crear una copia para experimentar

Los proyectos están en el repositorio `clases_javafx`. Haz una **copia** del proyecto 2 para no modificar el original:

```
git clone https://github.com/jgreau/clases_javafx.git
cd clases_javafx
cp -r 2_javafx-fxml laboratorio_javafx      # en Windows: copia y pega la carpeta y renómbrala
cd laboratorio_javafx
```

> No cambies nada más (ni `pom.xml` ni paquetes): vas a **agregar archivos nuevos** dentro de la misma estructura.

Paso 2 · Dónde van los archivos

Todos los ejemplos usan estas dos carpetas:

Tipo de archivo	Carpeta
`.java`	`src/main/java/com/duoc/javafxfxml/`
`.fxml` y `.css`	`src/main/resources/com/duoc/javafxfxml/`

> **Regla de oro:** el nombre y la carpeta tienen que coincidir exactamente con lo que dice cada ejemplo.

Paso 3 · El "visor" para probar cualquier pantalla

Para no escribir una clase `Application` por cada ejemplo, crea este **visor**. Abre el `.fxml` cuyo nombre le entregues al ejecutarlo.

Archivo: `src/main/java/com/duoc/javafxfxml/VisorFxml.java`

```
package com.duoc.javafxfxml;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;
import java.util.List;

/**
 * Visor de pruebas: abre cualquier archivo .fxml de la carpeta de recursos.
 * Uso: el primer argumento es el .fxml; el segundo (opcional) es un .css.
 */
public class VisorFxml extends Application {

    @Override
    public void start(Stage stage) throws IOException {
```

```
        List<String> args = getParameters().getRaw();
        String nombreFxml = args.isEmpty() ? "hola-view.fxml" : args.get(0);

        FXMLLoader loader = new FXMLLoader(getClass().getResource(nombreFxml));
        Parent raiz = loader.load();
        Scene scene = new Scene(raiz);

        if (args.size() > 1) {

scene.getStylesheets().add(getClass().getResource(args.get(1)).toExternalForm());
        }

        stage.setTitle("Visor: " + nombreFxml);
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Cómo funciona: `getParameters().getRaw()` entrega los argumentos que escribiste al ejecutar; el primero es el nombre del `.fxml` y el segundo (opcional) es un `.css`.

Paso 4 · Cómo ejecutar cualquier ejemplo

Línea de comandos (Linux / macOS / CMD):

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="layouts-view.fxml"
```

Windows PowerShell (los parámetros van entre comillas):

```
mvn javafx:run "-Dmain.class=com.duoc.javafxfxml.VisorFxml" "-Djavafx.args=layouts-
view.fxml"
```

Solo cambia el nombre del archivo (`layouts-view.fxml`, `controles-view.fxml`, ...).

IntelliJ: *Run → Edit Configurations... → + → Maven* y en **Command line** escribe:

```
javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args=layouts-view.fxml
```

Guárdala con un nombre claro (por ejemplo “Visor layouts”). Para otra pantalla, duplica la configuración y cambia el nombre del `.fxml`.

Prueba de que todo funciona

Ejecuta una vista que **ya existe** en tu proyecto:

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="hola-view.fxml"
```

Si ves la ventana del botón “Saludar”, tu laboratorio está listo. Si falla, revisa el Anexo D.

---

3. Etapa 1 · Componentes de UI en FXML

Qué es y para qué sirve

Un **componente** (o *nodo*) es cualquier elemento visual: un botón, una etiqueta, una tabla, un contenedor. En FXML cada componente es una **etiqueta XML** y sus propiedades son **atributos**:

```
<Button text="Guardar" onAction="#onGuardarClick" />
<!--    ↑ clase JavaFX    ↑ propiedad        ↑ evento        -->
```

Hay tres grandes familias:

Familia	Para qué	Ejemplos
Contenedores (layouts)	Ordenar otros componentes	`VBox`, `HBox`, `BorderPane`, `GridPane`, `StackPane`
Controles	Mostrar datos o pedirlos al usuario	`Label`, `Button`, `TextField`, `ComboBox`, `TableView`
Estructura de ventana	Organizar toda la pantalla	`MenuBar`, `TabPane`, `SplitPane`, `ToolBar`

> **Idea clave:** los componentes se **anidan** como un árbol. Un `BorderPane` contiene un `VBox`, que contiene botones. En FXML se ve como etiquetas dentro de etiquetas.

Etapa 1A · Layouts (cómo se ordena la pantalla)

Layout	Cómo ordena	Útil para
`VBox`	En **columna**	Formularios simples, menús laterales
`HBox`	En **fila**	Barras de botones
`BorderPane`	5 zonas: `top`, `left`, `center`, `right`, `bottom`	Estructura general de una ventana
`GridPane`	**Filas y columnas**	Formularios etiqueta + campo
`StackPane`	**Apilados** uno encima de otro	Centrar algo, superponer

Archivo: `src/main/resources/com/duoc/javafxfxml/layouts-view.fxml`

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.GridPane?>
```

```
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.Region?>
<?import javafx.scene.layout.VBox?>

<BorderPane prefHeight="360.0" prefWidth="560.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1">

   <!-- ARRIBA: un título -->
   <top>
      <Label text="Probando layouts" BorderPane.alignment="CENTER">
         <padding>
            <Insets bottom="10.0" top="10.0" />
         </padding>
      </Label>
   </top>

   <!-- IZQUIERDA: un VBox (botones en columna) -->
   <left>
      <VBox spacing="8.0" style="-fx-background-color: #eaf1f8;">
         <padding>
            <Insets bottom="12.0" left="12.0" right="12.0" top="12.0" />
         </padding>
         <children>
            <Button maxWidth="Infinity" text="Inicio" />
            <Button maxWidth="Infinity" text="Productos" />
            <Button maxWidth="Infinity" text="Ventas" />
         </children>
      </VBox>
   </left>

   <!-- CENTRO: un GridPane (formulario en filas y columnas) -->
   <center>
      <GridPane hgap="10.0" vgap="10.0">
         <padding>
            <Insets bottom="16.0" left="16.0" right="16.0" top="16.0" />
         </padding>
         <children>
            <Label text="Nombre:" GridPane.columnIndex="0" GridPane.rowIndex="0" />
            <TextField GridPane.columnIndex="1" GridPane.rowIndex="0" />
            <Label text="Correo:" GridPane.columnIndex="0" GridPane.rowIndex="1" />
            <TextField GridPane.columnIndex="1" GridPane.rowIndex="1" />
            <Button text="Guardar" GridPane.columnIndex="1" GridPane.rowIndex="2" />
         </children>
      </GridPane>
   </center>

   <!-- ABAJO: un HBox (fila) con un espacio flexible en medio -->
   <bottom>
      <HBox alignment="CENTER_LEFT" spacing="10.0" style="-fx-background-color: #dddddd;">
         <padding>
            <Insets bottom="8.0" left="12.0" right="12.0" top="8.0" />
         </padding>
         <children>
            <Label text="Estado: listo" />
            <Region HBox.hgrow="ALWAYS" />
            <Label text="v1.0" />
         </children>
      </HBox>
   </bottom>
</BorderPane>
```

Ejecútalo:

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="layouts-view.fxml"
```

Lo importante de este ejemplo

- `BorderPane` usa **zonas con nombre** (`<top>`, `<left>`, `<center>`, `<bottom>`).
- En `GridPane`, cada hijo indica su celda con `GridPane.columnIndex` y `GridPane.rowIndex`.
- `<Region HBox.hgrow="ALWAYS" />` es un **espacio flexible**: se estira y empuja a los demás hacia los extremos.
- `maxWidth="Infinity"` hace que los botones ocupen todo el ancho disponible.
- `<Insets>` dentro de `<padding>` crea margen **interior**; `spacing`, `hgap` y `vgap` separan los hijos.

Pruébalo

1. Cambia `spacing="8.0"` del `VBox` por `30.0`. ¿Qué cambia?
2. Quita `maxWidth="Infinity"` de un botón. ¿Qué pasa con su ancho?
3. Cambia `<Region HBox.hgrow="ALWAYS" />` por un `Label` con el texto "—". ¿Dónde queda ahora "v1.0"?
4. Agrega una **tercera fila** al `GridPane`: un `Label` "Edad:" y un `TextField` (usa `rowIndex="2"` y mueve el botón a `rowIndex="3"`).
5. Cambia `alignment="CENTER_LEFT"` del `HBox` inferior por `CENTER`.
6. Abre el archivo en **Scene Builder** y arrastra un `ListView` a la zona `right` del `BorderPane`.

Comprueba lo que aprendiste

> ▼ **¿Para qué sirve `HBox.hgrow="ALWAYS"` en un `Region`?**
>
> Hace que ese espacio vacío **se estire** para ocupar todo el espacio libre, empujando los otros componentes a los extremos.

> ▼ **¿Qué diferencia hay entre `padding` y `spacing`?**
>
> `padding` es el margen **interior** del contenedor (entre su borde y sus hijos). `spacing` es la separación **entre** los hijos.

Etapa 1B · Controles de formulario

Control	Dato que representa	Cómo lo lees	Evento principal
`TextField`	Texto de una línea	`getText()`	`onAction` (Enter)
`PasswordField`	Clave (oculta)	`getText()`	`onAction`
`TextArea`	Texto de varias líneas	`getText()`	—
`CheckBox`	Sí / No	`isSelected()`	`onAction`
`RadioButton` + `ToggleGroup`	Una opción entre varias	`getSelectedToggle()`	`onAction`

Control	Dato que representa	Cómo lo lees	Evento principal
`ComboBox<T>`	Una opción de una lista desplegable	`getValue()`	`onAction`
`Spinner<Integer>`	Número con flechas	`getValue()`	listener de `valueProperty()`
`Slider`	Número en un rango	`getValue()`	listener de `valueProperty()`
`DatePicker`	Fecha	`getValue()` (`LocalDate`, puede ser `null`)	`onAction`
`ProgressBar`	Avance (0.0 a 1.0)	`setProgress()`	—

Archivo: `src/main/resources/com/duoc/javafxfxml/controles-view.fxml`

```
<?xml version="1.0" encoding="UTF-8"?>

<?import java.lang.String?>
<?import javafx.collections.FXCollections?>
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.DatePicker?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.PasswordField?>
<?import javafx.scene.control.ProgressBar?>
<?import javafx.scene.control.RadioButton?>
<?import javafx.scene.control.Slider?>
<?import javafx.scene.control.Spinner?>
<?import javafx.scene.control.TextArea?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.control.ToggleGroup?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>

<VBox spacing="12.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.duoc.javafxfxml.ControlesController">
   <padding>
      <Insets bottom="16.0" left="16.0" right="16.0" top="16.0" />
   </padding>

   <!-- Un ToggleGroup hace que los RadioButton sean excluyentes -->
   <fx:define>
      <ToggleGroup fx:id="generoGroup" />
   </fx:define>

   <children>
      <Label style="-fx-font-size: 16px; -fx-font-weight: bold;" text="Catálogo de
controles" />

      <GridPane hgap="10.0" vgap="8.0">
         <children>
            <Label text="Nombre" GridPane.rowIndex="0" />
```

```
            <TextField fx:id="nombreField" promptText="Tu nombre" GridPane.columnIndex="1"
GridPane.rowIndex="0" />

            <Label text="Clave" GridPane.rowIndex="1" />
            <PasswordField fx:id="claveField" promptText="Clave" GridPane.columnIndex="1"
GridPane.rowIndex="1" />

            <Label text="Género" GridPane.rowIndex="2" />
            <HBox spacing="10.0" GridPane.columnIndex="1" GridPane.rowIndex="2">
               <children>
                  <RadioButton text="Femenino" toggleGroup="$generoGroup" />
                  <RadioButton text="Masculino" toggleGroup="$generoGroup" />
                  <RadioButton text="Otro" toggleGroup="$generoGroup" />
               </children>
            </HBox>

            <Label text="Carrera" GridPane.rowIndex="3" />
            <ComboBox fx:id="carreraCombo" promptText="Elige una" GridPane.columnIndex="1"
GridPane.rowIndex="3">
               <items>
                  <FXCollections fx:factory="observableArrayList">
                     <String fx:value="Informática" />
                     <String fx:value="Administración" />
                     <String fx:value="Diseño" />
                  </FXCollections>
               </items>
            </ComboBox>

            <Label text="Edad" GridPane.rowIndex="4" />
            <Spinner fx:id="edadSpinner" editable="true" initialValue="20" max="99"
min="16" GridPane.columnIndex="1" GridPane.rowIndex="4" />

            <Label text="Nacimiento" GridPane.rowIndex="5" />
            <DatePicker fx:id="nacimientoPicker" GridPane.columnIndex="1"
GridPane.rowIndex="5" />

            <Label text="Nivel (0-100)" GridPane.rowIndex="6" />
            <VBox spacing="4.0" GridPane.columnIndex="1" GridPane.rowIndex="6">
               <children>
                  <Slider fx:id="nivelSlider" majorTickUnit="25.0" max="100.0"
showTickLabels="true" showTickMarks="true" value="50.0" />
                  <ProgressBar fx:id="nivelProgress" maxWidth="Infinity" />
               </children>
            </VBox>

            <Label text="Comentarios" GridPane.rowIndex="7" />
            <TextArea fx:id="comentariosArea" prefRowCount="3" wrapText="true"
GridPane.columnIndex="1" GridPane.rowIndex="7" />
         </children>
      </GridPane>

      <CheckBox fx:id="terminosCheck" text="Acepto los términos" />
      <Button onAction="#onResumenClick" text="Ver resumen" />
      <TextArea fx:id="resumenArea" editable="false" prefRowCount="6" />
   </children>
</VBox>
```

Archivo: `src/main/java/com/duoc/javafxfxml/ControlesController.java`

```
package com.duoc.javafxfxml;

import javafx.fxml.FXML;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.DatePicker;
import javafx.scene.control.PasswordField;
import javafx.scene.control.ProgressBar;
import javafx.scene.control.RadioButton;
import javafx.scene.control.Slider;
import javafx.scene.control.Spinner;
import javafx.scene.control.TextArea;
import javafx.scene.control.TextField;
import javafx.scene.control.Toggle;
import javafx.scene.control.ToggleGroup;

import java.time.LocalDate;

public class ControlesController {

    @FXML private TextField nombreField;
    @FXML private PasswordField claveField;
    @FXML private ToggleGroup generoGroup;
    @FXML private ComboBox<String> carreraCombo;
    @FXML private Spinner<Integer> edadSpinner;
    @FXML private DatePicker nacimientoPicker;
    @FXML private Slider nivelSlider;
    @FXML private ProgressBar nivelProgress;
    @FXML private TextArea comentariosArea;
    @FXML private CheckBox terminosCheck;
    @FXML private TextArea resumenArea;

    @FXML
    private void initialize() {
        // La barra de progreso siempre refleja el valor del slider (0.0 a 1.0)
        nivelProgress.progressProperty().bind(nivelSlider.valueProperty().divide(100));
    }

    @FXML
    private void onResumenClick() {
        Toggle seleccionado = generoGroup.getSelectedToggle();
        String genero = (seleccionado == null) ? "(sin elegir)" : ((RadioButton)
seleccionado).getText();

        String carrera = carreraCombo.getValue() == null ? "(sin elegir)" :
carreraCombo.getValue();
        LocalDate nacimiento = nacimientoPicker.getValue();

        resumenArea.setText(
                "Nombre: " + nombreField.getText() + "\n"
                + "Largo de la clave: " + claveField.getText().length() + "\n"
                + "Género: " + genero + "\n"
                + "Carrera: " + carrera + "\n"
                + "Edad: " + edadSpinner.getValue() + "\n"
                + "Nacimiento: " + (nacimiento == null ? "(sin elegir)" : nacimiento) +
"\n"
                + "Nivel: " + (int) nivelSlider.getValue() + "\n"
                + "Comentarios: " + comentariosArea.getText() + "\n"
```

```
                + "Acepta términos: " + (terminosCheck.isSelected() ? "sí" : "no"));
    }
}
```

Ejecútalo:

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="controles-
view.fxml"
```

Lo importante de este ejemplo

- **`<fx:define>` + `ToggleGroup`:** define un objeto que no se dibuja. Los `RadioButton` lo referencian con `toggleGroup="$generoGroup"` (el `$` significa "usa el objeto con ese `fx:id`"). Así solo uno puede estar seleccionado.
- **`<items>` con `FXCollections`:** llena el `ComboBox` directamente desde el FXML.
- **`Spinner`:** `min`, `max` e `initialValue` configuran el rango. Con `editable="true"` se puede escribir.
- **`@FXML private ToggleGroup generoGroup;`:** el controlador recibe el `ToggleGroup` igual que cualquier otro `fx:id`.
- **`getSelectedToggle()`** puede devolver `null` si no se eligió nada: siempre hay que revisarlo.
- **Binding:** `nivelProgress.progressProperty().bind(nivelSlider.valueProperty().divide(100))` hace que la barra siga al slider **sin escribir ningún evento**.

Pruébalo

1. Ejecuta, llena el formulario y presiona **Ver resumen**. Luego presiónalo **sin llenar nada**: ¿qué muestra para el género y la carrera? ¿Por qué no se cae?
2. Mueve el `Slider` y observa la `ProgressBar`. Luego **comenta** la línea del `bind` en `initialize()` y vuelve a probar.
3. Agrega una cuarta carrera (`<String fx:value="Contabilidad" />`).
4. Cambia el rango del `Spinner` a `min="18"` y `max="60"`.
5. Agrega un `RadioButton` "Prefiero no decir".
6. **Provoca un error:** cambia el `fx:id` de `nombreField` por `nombre` en el FXML (sin tocar el controlador) y ejecuta. Lee el mensaje de error: ¿qué te dice?

Comprueba lo que aprendiste

> **▼ ¿Por qué los `RadioButton` necesitan un `ToggleGroup`?**
>
> Para que sean **excluyentes**: al elegir uno, se desmarca el anterior. Sin grupo, cada uno funciona por separado.

> **▼ ¿Cómo sé cuál `RadioButton` está elegido?**
>
> Con `grupo.getSelectedToggle()`. Devuelve un `Toggle` (hay que convertirlo a `RadioButton` para leer su texto) o `null` si no hay ninguno elegido.

▼ **¿Qué diferencia hay entre `bind` y escribir un listener?**

`bind` conecta dos propiedades de forma **declarativa** (una siempre sigue a la otra). Un listener es **código** que tú escribes para ejecutar cuando algo cambia. `bind` es más corto cuando solo quieres que un valor refleje a otro.

Etapa 1C · Contenedores avanzados: menús, pestañas, listas y tablas

Componente	Para qué
`MenuBar` → `Menu` → `MenuItem`	Menú superior (Archivo, Ayuda…)
`ToolBar`	Barra de herramientas con botones
`TabPane` → `Tab`	Pestañas
`SplitPane`	Dos paneles con divisor movible
`Accordion` → `TitledPane`	Secciones que se despliegan
`ListView<T>`	Lista de elementos seleccionables
`TableView<T>` → `TableColumn`	Tabla de datos
`Alert`	Cuadros de diálogo (información, error, confirmación)

Archivo: `src/main/resources/com/duoc/javafxfxml/contenedores-view.fxml`

```
<?xml version="1.0" encoding="UTF-8"?>

<?import java.lang.String?>
<?import javafx.collections.FXCollections?>
<?import javafx.scene.control.Accordion?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.ListView?>
<?import javafx.scene.control.Menu?>
<?import javafx.scene.control.MenuBar?>
<?import javafx.scene.control.MenuItem?>
<?import javafx.scene.control.SeparatorMenuItem?>
<?import javafx.scene.control.SplitPane?>
<?import javafx.scene.control.Tab?>
<?import javafx.scene.control.TabPane?>
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.control.TextArea?>
<?import javafx.scene.control.TitledPane?>
<?import javafx.scene.control.ToolBar?>
<?import javafx.scene.control.cell.PropertyValueFactory?>
<?import javafx.scene.layout.BorderPane?>

<BorderPane prefHeight="380.0" prefWidth="600.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.duoc.javafxfxml.ContenedoresController">

   <top>
      <MenuBar>
         <menus>
```

```
            <Menu text="Archivo">
               <items>
                  <MenuItem onAction="#onNuevoClick" text="Nuevo" />
                  <SeparatorMenuItem />
                  <MenuItem onAction="#onSalirClick" text="Salir" />
               </items>
            </Menu>
            <Menu text="Ayuda">
               <items>
                  <MenuItem onAction="#onAcercaClick" text="Acerca de..." />
               </items>
            </Menu>
         </menus>
      </MenuBar>
   </top>

   <center>
      <TabPane tabClosingPolicy="UNAVAILABLE">
         <tabs>
            <Tab text="Lista + detalle">
               <content>
                  <SplitPane dividerPositions="0.35">
                     <items>
                        <ListView fx:id="listaView">
                           <items>
                              <FXCollections fx:factory="observableArrayList">
                                 <String fx:value="Manzana" />
                                 <String fx:value="Pera" />
                                 <String fx:value="Uva" />
                              </FXCollections>
                           </items>
                        </ListView>
                        <TextArea fx:id="detalleArea" editable="false" text="Selecciona un
elemento de la lista" wrapText="true" />
                     </items>
                  </SplitPane>
               </content>
            </Tab>

            <Tab text="Tabla">
               <content>
                  <TableView fx:id="personasTable">
                     <columns>
                        <TableColumn prefWidth="200.0" text="Nombre">
                           <cellValueFactory><PropertyValueFactory property="nombre" />
</cellValueFactory>
                        </TableColumn>
                        <TableColumn prefWidth="80.0" text="Edad">
                           <cellValueFactory><PropertyValueFactory property="edad" />
</cellValueFactory>
                        </TableColumn>
                        <TableColumn prefWidth="150.0" text="Ciudad">
                           <cellValueFactory><PropertyValueFactory property="ciudad" />
</cellValueFactory>
                        </TableColumn>
                     </columns>
                  </TableView>
               </content>
            </Tab>

            <Tab text="Acordeón">
```

```
               <content>
                  <Accordion>
                     <panes>
                        <TitledPane text="Sección 1">
                           <content><Label text="Contenido de la sección 1" /></content>
                        </TitledPane>
                        <TitledPane text="Sección 2">
                           <content><Label text="Contenido de la sección 2" /></content>
                        </TitledPane>
                     </panes>
                  </Accordion>
               </content>
            </Tab>
         </tabs>
      </TabPane>
   </center>

   <bottom>
      <ToolBar>
         <items>
            <Button onAction="#onNuevoClick" text="Nuevo" />
            <Label fx:id="estadoLabel" text="Listo" />
         </items>
      </ToolBar>
   </bottom>
</BorderPane>
```

Archivo: `src/main/java/com/duoc/javafxfxml/Persona.java`

```
package com.duoc.javafxfxml;

/** Clase simple para llenar la tabla. PropertyValueFactory busca los getters (getNombre,
getEdad...). */
public class Persona {

    private final String nombre;
    private final int edad;
    private final String ciudad;

    public Persona(String nombre, int edad, String ciudad) {
        this.nombre = nombre;
        this.edad = edad;
        this.ciudad = ciudad;
    }

    public String getNombre() { return nombre; }

    public int getEdad() { return edad; }

    public String getCiudad() { return ciudad; }
}
```

Archivo: `src/main/java/com/duoc/javafxfxml/ContenedoresController.java`

```
package com.duoc.javafxfxml;

import javafx.application.Platform;
import javafx.collections.FXCollections;
import javafx.fxml.FXML;
```

```
import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.scene.control.ListView;
import javafx.scene.control.TableView;
import javafx.scene.control.TextArea;

public class ContenedoresController {

    @FXML private ListView<String> listaView;
    @FXML private TextArea detalleArea;
    @FXML private TableView<Persona> personasTable;
    @FXML private Label estadoLabel;

    @FXML
    private void initialize() {
        // Datos de ejemplo para la tabla
        personasTable.setItems(FXCollections.observableArrayList(
                new Persona("Ana Pérez", 21, "Santiago"),
                new Persona("Luis Soto", 25, "Valparaíso"),
                new Persona("Eva Rojas", 19, "Concepción")));

        // Reaccionar a la selección de la lista (se ve en detalle en la etapa de eventos)
        listaView.getSelectionModel().selectedItemProperty().addListener((obs, anterior,
actual) -> {
            if (actual != null) {
                detalleArea.setText("Elegiste: " + actual);
            }
        });
    }

    @FXML
    private void onNuevoClick() {
        estadoLabel.setText("Hiciste clic en Nuevo");
    }

    @FXML
    private void onSalirClick() {
        Platform.exit();
    }

    @FXML
    private void onAcercaClick() {
        Alert alerta = new Alert(Alert.AlertType.INFORMATION);
        alerta.setTitle("Acerca de");
        alerta.setHeaderText("Laboratorio JavaFX");
        alerta.setContentText("Ejemplo de menús, pestañas, tablas y diálogos.");
        alerta.showAndWait();
    }
}
```

Ejecútalo:

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="contenedores-
view.fxml"
```

Lo importante de este ejemplo

- **`TableView`:** cada `TableColumn` tiene un `PropertyValueFactory` con el nombre de una propiedad. JavaFX busca el *getter* con ese nombre: `property="nombre"` → `getNombre()`.

Si el getter no existe o se llama distinto, la columna queda vacía.

- **`MenuItem` con `onAction`:** los menús también disparan eventos de acción.
- **`Alert`:** no se define en FXML, se crea en Java. `showAndWait()` detiene el programa hasta que el usuario cierra el cuadro.
- **`setItems` / `observableArrayList`:** la tabla muestra lo que haya en la lista; si agregas un elemento, aparece solo.

Pruébalo

1. Selecciona elementos en la lista de la pestaña 1: el área de la derecha cambia (viene de un listener, lo vemos en la etapa 2).
2. Agrega una cuarta persona en `initialize()` y ejecuta.
3. Agrega una columna **"Apellido"**: necesitas el atributo, el constructor, el *getter* `getApellido()` y la `TableColumn` con `property="apellido"`.
4. **Provoca un error:** cambia `property="ciudad"` por `property="Ciudad"` (mayúscula). ¿Qué pasa con la columna? ¿Por qué?
5. Agrega un menú **Editar** con la opción **Copiar** que escriba "Copiaste" en `estadoLabel`.
6. Cambia el tipo de `Alert` a `WARNING` y luego a `CONFIRMATION`.

Comprueba lo que aprendiste

> ▼ **¿Por qué la columna queda vacía si el *getter* tiene otro nombre?**
>
> `PropertyValueFactory("ciudad")` busca el método `getCiudad()` (o `ciudadProperty()`) por **convención de nombres**. Si no lo encuentra, no muestra nada y no lanza error.

> ▼ **¿Qué hace `showAndWait()` en un `Alert`?**
>
> Muestra el cuadro y **bloquea** la ejecución en ese punto hasta que el usuario lo cierra. Devuelve el botón que se presionó.

★ Reto de la Etapa 1 · Tarjeta de producto

Diseña **en Scene Builder** (sin código Java, o con un controlador vacío) una pantalla de producto con:

1. Un `BorderPane` como raíz.
2. Arriba: un título y un `MenuBar` con *Archivo* → *Salir*.
3. A la izquierda: un `VBox` con 3 botones de categorías.
4. Al centro: un `GridPane` con **código**, **nombre**, **precio** (`Spinner`), **categoría** (`ComboBox`) y **disponible** (`CheckBox`).
5. Abajo: un `HBox` con el botón **Guardar** a la derecha (usa un `Region` flexible) y un `Label` de estado a la izquierda.

Autoevaluación: ☐ se ve ordenada al agrandar la ventana · ☐ cada campo tiene su `fx:id` · ☐ usaste al menos 3 layouts distintos · ☐ la abres con el visor sin errores.

> ✎ **Foro:** publica una **captura** de tu tarjeta y cuenta qué layout te costó más y por qué.

---

4. Etapa 2 · Eventos de usuario

Qué es y para qué sirve

Un **evento** es algo que ocurre en la interfaz: un clic, una tecla, un movimiento del mouse, un cambio de valor. Tu programa no "espera en un ciclo": **registras un método** (*manejador* o *handler*) y JavaFX lo **llama cuando el evento ocurre**. Esto se llama **programación orientada a eventos**.

```
Usuario hace clic  →  JavaFX crea un ActionEvent  →  llama a tu método onGuardarClick()
```

Tres formas de conectar un evento

Forma	Ejemplo	Cuándo usarla
En el FXML	`<Button onAction="#onGuardarClick" />`	Lo más habitual con FXML
En Java con método	`boton.setOnAction(e -> guardar());`	Controles creados por código
Listener de propiedad	`slider.valueProperty().addListener(...)`	Reaccionar a *cambios de valor*

Tipos de evento más usados

Evento	Se dispara cuando…	Atributo FXML	Objeto del evento
Acción	Clic en botón, Enter en un campo, cambio en CheckBox/ComboBox	`onAction`	`ActionEvent`
Clic de mouse	Se hace clic	`onMouseClicked`	`MouseEvent`
Mouse entra / sale	El puntero entra o sale del componente	`onMouseEntered` / `onMouseExited`	`MouseEvent`
Mouse se mueve	El puntero se mueve dentro	`onMouseMoved`	`MouseEvent`
Tecla presionada	Se presiona una tecla	`onKeyPressed`	`KeyEvent`
Tecla escrita	Se escribe un carácter	`onKeyTyped`	`KeyEvent`

El objeto del evento

Tu método puede **recibir el evento como parámetro** y obtener información de él:

Método	Devuelve	Ejemplo de uso
`evento.getSource()`	El componente que disparó el evento	Saber **qué botón** fue

Método	Devuelve	Ejemplo de uso
`evento.getX()` / `getY()`	Posición del mouse en el componente	Dibujar donde se hace clic
`evento.getButton()`	`PRIMARY` (izq.), `SECONDARY` (der.), …	Distinguir clic derecho
`evento.getClickCount()`	Cantidad de clics seguidos	Detectar doble clic
`evento.getCode()`	Una constante `KeyCode` (`ENTER`, `ESCAPE`…)	Atajos de teclado
`evento.getCharacter()`	El carácter escrito (solo en `onKeyTyped`)	Filtrar lo que se escribe
`evento.isShiftDown()`	`true` si Shift está apretado	Combinaciones de teclas
`evento.consume()`	—	Detener que el evento siga “subiendo”

Archivo: `src/main/resources/com/duoc/javafxfxml/eventos-view.fxml`

```
<?xml version="1.0" encoding="UTF-8"?>

<?import java.lang.String?>
<?import javafx.collections.FXCollections?>
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.CheckBox?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.Slider?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.StackPane?>
<?import javafx.scene.layout.VBox?>

<VBox fx:id="raiz" prefWidth="480.0" spacing="12.0" onKeyPressed="#onTeclaEnRaiz"
xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.duoc.javafxfxml.EventosController">
   <padding>
      <Insets bottom="16.0" left="16.0" right="16.0" top="16.0" />
   </padding>
   <children>
      <Label style="-fx-font-size: 16px; -fx-font-weight: bold;" text="Laboratorio de
eventos" />

      <!-- 1. Evento onAction (clic o Enter en un botón) -->
      <HBox alignment="CENTER_LEFT" spacing="10.0">
         <children>
            <Button onAction="#onContarClick" text="Contar clics" />
            <Label fx:id="contadorLabel" text="Clics: 0" />
         </children>
      </HBox>

      <!-- 2. Eventos de mouse sobre una zona -->
      <StackPane fx:id="zonaMouse" minHeight="70.0" onMouseClicked="#onZonaClick"
```

```
onMouseEntered="#onZonaEntra" onMouseExited="#onZonaSale" onMouseMoved="#onZonaMueve"
style="-fx-background-color: #eaf1f8; -fx-border-color: #1f4e79;">
         <children>
            <Label fx:id="zonaLabel" text="Mueve el mouse aquí, haz clic o doble clic" />
         </children>
      </StackPane>

      <!-- 3. Eventos de teclado -->
      <TextField fx:id="tecladoField" onAction="#onEnterEnCampo"
onKeyPressed="#onTeclaPresionada" onKeyTyped="#onTeclaEscrita" promptText="Escribe aquí
(Enter envía, Esc limpia)" />
      <Label fx:id="tecladoLabel" text="Última tecla: -" />

      <!-- 4. Eventos de controles: CheckBox, ComboBox, Slider -->
      <HBox alignment="CENTER_LEFT" spacing="12.0">
         <children>
            <CheckBox fx:id="notificarCheck" onAction="#onNotificarCambio"
text="Notificarme" />
            <ComboBox fx:id="colorCombo" onAction="#onColorElegido" promptText="Color">
               <items>
                  <FXCollections fx:factory="observableArrayList">
                     <String fx:value="Rojo" />
                     <String fx:value="Verde" />
                     <String fx:value="Azul" />
                  </FXCollections>
               </items>
            </ComboBox>
         </children>
      </HBox>
      <Slider fx:id="volumenSlider" max="100.0" showTickLabels="true" value="30.0" />
      <Label fx:id="controlesLabel" text="Cambia un control y mira qué pasa" />

      <!-- 5. Un solo método para varios botones: se distingue con getSource() -->
      <HBox spacing="8.0">
         <children>
            <Button onAction="#onBotonColor" text="Rojo" userData="#c62828" />
            <Button onAction="#onBotonColor" text="Verde" userData="#2e7d32" />
            <Button onAction="#onBotonColor" text="Azul" userData="#1565c0" />
         </children>
      </HBox>
      <Label fx:id="colorLabel" text="Presiona un botón de color" />
   </children>
</VBox>
```

Archivo: `src/main/java/com/duoc/javafxfxml/EventosController.java`

```
package com.duoc.javafxfxml;

import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.control.Slider;
import javafx.scene.control.TextField;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.input.MouseButton;
import javafx.scene.input.MouseEvent;
```

```
import javafx.event.ActionEvent;

public class EventosController {

    @FXML private Label contadorLabel;
    @FXML private Label zonaLabel;
    @FXML private TextField tecladoField;
    @FXML private Label tecladoLabel;
    @FXML private CheckBox notificarCheck;
    @FXML private ComboBox<String> colorCombo;
    @FXML private Slider volumenSlider;
    @FXML private Label controlesLabel;
    @FXML private Label colorLabel;

    private int clics = 0;

    @FXML
    private void initialize() {
        // Evento "de propiedad": se dispara cada vez que cambia el valor (no solo con el
mouse)
        volumenSlider.valueProperty().addListener((observable, valorAnterior, valorNuevo) -
>
                controlesLabel.setText("Volumen: " + valorNuevo.intValue()));
    }

    // ---------- 1. onAction ----------
    @FXML
    private void onContarClick() {
        clics++;
        contadorLabel.setText("Clics: " + clics);
    }

    // ---------- 2. Mouse ----------
    @FXML
    private void onZonaClick(MouseEvent evento) {
        String boton = evento.getButton() == MouseButton.PRIMARY ? "izquierdo"
                : evento.getButton() == MouseButton.SECONDARY ? "derecho" : "otro";
        zonaLabel.setText("Clic " + boton + " ×" + evento.getClickCount()
                + " en (" + (int) evento.getX() + ", " + (int) evento.getY() + ")");
    }

    @FXML
    private void onZonaEntra(MouseEvent evento) {
        ((Node) evento.getSource()).setStyle("-fx-background-color: #cfe3f7; -fx-border-
color: #1f4e79;");
    }

    @FXML
    private void onZonaSale(MouseEvent evento) {
        ((Node) evento.getSource()).setStyle("-fx-background-color: #eaf1f8; -fx-border-
color: #1f4e79;");
        zonaLabel.setText("Mueve el mouse aquí, haz clic o doble clic");
    }

    @FXML
    private void onZonaMueve(MouseEvent evento) {
        zonaLabel.setText("Posición: " + (int) evento.getX() + ", " + (int) evento.getY());
    }

    // ---------- 3. Teclado ----------
    @FXML
```

```
    private void onTeclaPresionada(KeyEvent evento) {
        tecladoLabel.setText("Última tecla: " + evento.getCode()
                + (evento.isShiftDown() ? " (con Shift)" : ""));
    }

    @FXML
    private void onTeclaEscrita(KeyEvent evento) {
        // getCharacter() entrega el carácter escrito (solo existe en KEY_TYPED)
        System.out.println("Carácter escrito: '" + evento.getCharacter() + "'");
    }

    @FXML
    private void onEnterEnCampo() {
        tecladoLabel.setText("Enviaste: " + tecladoField.getText());
        tecladoField.clear();
    }

    // El evento de teclado "sube" por el árbol (bubbling): llega a la raíz aunque el foco
esté en el campo
    @FXML
    private void onTeclaEnRaiz(KeyEvent evento) {
        if (evento.getCode() == KeyCode.ESCAPE) {
            tecladoField.clear();
            tecladoLabel.setText("Campo limpiado con Esc");
        }
    }

    // ---------- 4. Controles ----------
    @FXML
    private void onNotificarCambio() {
        controlesLabel.setText("Notificaciones: " + (notificarCheck.isSelected() ?
"activadas" : "desactivadas"));
    }

    @FXML
    private void onColorElegido() {
        controlesLabel.setText("Color elegido: " + colorCombo.getValue());
    }

    // ---------- 5. Un método, varios botones ----------
    @FXML
    private void onBotonColor(ActionEvent evento) {
        Button boton = (Button) evento.getSource();     // ¿cuál botón fue?
        String codigoColor = (String) boton.getUserData();
        colorLabel.setText("Elegiste " + boton.getText());
        colorLabel.setStyle("-fx-text-fill: " + codigoColor + "; -fx-font-weight: bold;");
    }
}
```

Ejecútalo:

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="eventos-view.fxml"
```

Lo importante de este ejemplo

1. **`onAction`**: el método sin parámetros es válido (`onContarClick()`). Se dispara con el mouse **y también con teclado** (Enter/Espacio sobre el botón enfocado).

2. **Eventos de mouse**: el método recibe un `MouseEvent`. Fíjate en cómo se distingue clic izquierdo, derecho y doble clic con `getButton()` y `getClickCount()`.
3. **`getSource()`**: en `onZonaEntra` se usa para cambiar el estilo del componente que disparó el evento, sin necesidad de un `@FXML` para él.
4. **Teclado**: `onKeyPressed` entrega `getCode()`; `onKeyTyped` entrega `getCharacter()` (mira la consola).
5. **Propagación (*bubbling*)**: el `VBox` raíz tiene `onKeyPressed`. Aunque el foco esté en el `TextField`, el evento de teclado **sube** por el árbol y llega a la raíz, por eso **Esc limpia el campo**.
6. **Listener**: en `initialize()` se registra `volumenSlider.valueProperty().addListener(...)`. Recibe tres cosas: la propiedad observada, el valor **anterior** y el valor **nuevo**.
7. **Un método para varios botones**: `onBotonColor` sirve para tres botones; `getSource()` indica cuál fue, y `getUserData()` guarda un dato asociado a cada uno.

Pruébalo

1. Presiona **Contar clics** varias veces. Luego enfócalo con `Tab` y presiona **Espacio** o **Enter**: ¿cuenta? (`onAction` no es solo del mouse).
2. En la zona azul: mueve el mouse, haz **clic izquierdo, derecho y doble clic**. Anota qué texto aparece en cada caso.
3. Escribe en el campo de texto y mira la **consola**: ¿qué imprime `onKeyTyped` al escribir una mayúscula? ¿Y al presionar Shift?
4. Presiona **Enter** en el campo. Luego escribe algo y presiona **Esc**.
5. Cambia el `ComboBox` y el `CheckBox`, y mueve el slider. ¿Qué escribe el `Label` en cada caso?
6. **Experimento (bubbling):** en `onTeclaPresionada` (el manejador del `TextField`), agrega `evento.consume();` como última línea. Ahora, con el foco en el campo, presiona **Esc**: ¿sigue limpiándose? ¿Por qué? Quita el `consume()` y vuelve a probar.
7. **Agrega un evento:** en el `StackPane`, agrega `onMousePressed="#onZonaPresiona"` y crea el método que escriba “Presionaste” en `zonaLabel`. Prueba cuándo se dispara comparado con `onMouseClicked`.
8. **Agrega un botón “Rosado”** con otro color en `userData` **sin tocar el controlador**. ¿Funciona? ¿Por qué?

Comprueba lo que aprendiste

<details>
<summary>¿Cuál es la diferencia entre <code>onAction</code> y <code>onMouseClicked</code> en un botón?</summary>

`onAction` se dispara por **cualquier forma** de activar el botón (mouse, Enter o Espacio con el botón enfocado). `onMouseClicked` solo ocurre con el **mouse**.
</details>

<details>
<summary>¿Cómo sé cuál botón disparó un método compartido por varios?</summary>

Con `evento.getSource()`, que devuelve el componente. Hay que convertirlo (`(Button) evento.getSource()`) para usar sus métodos propios.
</details>

▼ **¿Por qué `Esc` funciona aunque el foco esté en el `TextField` y el manejador esté en el `VBox`?**

Por la **propagación** (*bubbling*): el evento nace en el componente con foco y luego **sube** por sus padres. El `VBox` lo recibe en el camino.

▼ **¿Cuándo usar un listener en vez de `onAction`?**

Cuando quieres reaccionar a **cualquier cambio de valor** (por ejemplo, el slider mientras se arrastra) y no a una acción puntual del usuario. El listener entrega el valor anterior y el nuevo.

★ Reto de la Etapa 2 · Contador con teclado

Crea `contador-view.fxml` + `ContadorController` con:

1. Un `Label` grande que muestra un número (parte en 0).
2. Botones **+**, **−** y **Reiniciar**, que comparten **un solo método** y se distinguen con `getSource()` o `userData`.
3. Con el teclado: ↑ suma, ↓ resta, **R** reinicia (`KeyCode.UP`, `KeyCode.DOWN`, `KeyCode.R`).
4. El contador **no puede bajar de 0** ni subir de 20. Al llegar al límite, mostrar un mensaje en otro `Label`.
5. Un `Slider` de 0 a 20 que **siempre** muestra el mismo valor que el contador (usa un listener o `bind`).
6. Un doble clic sobre el número lo reinicia a 0.

Pista: el manejo de teclado en una pantalla que no tiene un campo de texto necesita que algún componente tenga el **foco**; piensa dónde conviene poner el `onKeyPressed`.

Autoevaluación: ☐ cada tipo de evento (`onAction`, teclado, mouse, listener) aparece al menos una vez · ☐ los límites se respetan siempre · ☐ la lógica del límite no está repetida en tres lugares.

✎ **Foro:** cuenta **dónde pusiste la lógica de los límites** (en cada manejador o en un método aparte) y por qué. Comenta la decisión de un compañero.

---

5. Etapa 3 · Estilos CSS en JavaFX

Qué es y para qué sirve

CSS (*Cascading Style Sheets*) permite definir **cómo se ve** la interfaz en un archivo aparte, sin tocar el código Java ni el FXML. Así separas **estructura** (FXML), **lógica** (Java) y **apariencia** (CSS).

JavaFX usa su propio CSS: casi igual al de la web, pero **las propiedades empiezan con `-fx-`** y tienen nombres propios.

```
.boton-primario {
    -fx-background-color: #1f4e79;   /* color de fondo */
    -fx-text-fill: white;            /* color del texto (¡no es "color"!) */
}
```

Tres formas de aplicar estilos

Forma	Ejemplo	Cuándo usarla
Estilo en línea	`<Button style="-fx-text-fill: red;" />` o `boton.setStyle("...")`	Cambios puntuales y rápidos
Hoja de estilos en el FXML	`<VBox stylesheets="@estilos.css">`	Una pantalla con su propio estilo
Hoja de estilos en la escena	`scene.getStylesheets().add(...)`	Estilo global de toda la aplicación

> **Recomendación:** usa **hojas de estilo** (`.css`). El estilo en línea es difícil de mantener porque queda mezclado con el código.

Los selectores: a quién se le aplica el estilo

Selector	Escribe	Se aplica a…	Cómo se asigna
Tipo	`.button`, `.label`, `.text-field`	Todos los controles de ese tipo	Automático (nombre del control)
Clase	`.boton-primario`	Los que tengan esa clase	`styleClass="boton-primario"` en FXML o `getStyleClass().add(...)`
Id	`#mensajeLabel`	El componente con ese `fx:id`	Con el `fx:id`
Pseudo-clase	`.button:hover`	Un **estado** del componente	Automático (el estado actual)
Raíz	`.root`	La raíz de la escena	Automático

Pseudo-clases más útiles: `:hover` (mouse encima), `:pressed` (presionado), `:focused` (con foco), `:disabled` (deshabilitado), `:selected` (seleccionado).

> **Nombres de estilo de los controles:** se escriben en minúscula y con guion: `Button` → `.button`, `TextField` → `.text-field`, `ComboBox` → `.combo-box`, `TableView` → `.table-view`, `CheckBox` → `.check-box`.

Propiedades más usadas

(Ver tabla completa en el Anexo C.)

Propiedad	Para qué	Ejemplo
`-fx-background-color`	Color de fondo	`#ffffff`, `red`, `rgba(0,0,0,0.5)`
`-fx-text-fill`	Color del texto	`white`
`-fx-font-size` / `-fx-font-weight`	Tamaño / grosor	`18px` / `bold`
`-fx-padding`	Margen interior	`8 16 8 16` (arriba derecha abajo izquierda)
`-fx-background-radius`	Esquinas redondeadas del fondo	`6`
`-fx-border-color` / `-fx-border-width` / `-fx-border-radius`	Borde	`red` / `2` / `4`
`-fx-effect`	Sombras	`dropshadow(gaussian, rgba(0,0,0,0.2), 10, 0, 0, 2)`
`-fx-cursor`	Cursor del mouse	`hand`

Variables de color

En JavaFX se pueden definir **colores con nombre** en `.root` y reutilizarlos. Si cambias la variable, cambia **todo** lo que la usa. Es la base del tema oscuro.

```
.root { -color-primario: #1f4e79; }
.boton { -fx-background-color: -color-primario; }
```

Para variar un color puedes usar `derive(color, porcentaje)`: positivo aclara, negativo oscurece (`derive(-color-primario, 25%)`).

Laboratorio de estilos

Archivo: `src/main/resources/com/duoc/javafxfxml/estilos.css`

```
/* ===== Variables de color (se definen en .root y se usan en todo el archivo) ===== */
.root {
    -color-primario: #1f4e79;
    -color-fondo: #f4f7fb;
    -color-texto: #222222;
    -color-error: #c62828;
    -color-ok: #2e7d32;

    -fx-background-color: -color-fondo;
}

/* ===== Selector de CLASE: .nombre  (se asigna con styleClass) ===== */
.titulo {
```

```
    -fx-font-size: 22px;
    -fx-font-weight: bold;
    -fx-text-fill: -color-primario;
}

.tarjeta {
    -fx-background-color: white;
    -fx-background-radius: 10;
    -fx-padding: 16;
    -fx-effect: dropshadow(gaussian, rgba(0, 0, 0, 0.15), 10, 0, 0, 2);
}

.boton-primario {
    -fx-background-color: -color-primario;
    -fx-text-fill: white;
    -fx-background-radius: 6;
    -fx-padding: 8 16 8 16;
    -fx-cursor: hand;
}

/* ===== PSEUDO-CLASES: estados del componente (:hover, :pressed, :disabled, :focused)
===== */
.boton-primario:hover {
    -fx-background-color: derive(-color-primario, 25%);
}

.boton-primario:pressed {
    -fx-background-color: derive(-color-primario, -20%);
}

.boton-primario:disabled {
    -fx-opacity: 0.5;
}

/* ===== Selector de TIPO: usa el nombre de estilo del control (.text-field, .label,
.button...) ===== */
.text-field:focused {
    -fx-border-color: -color-primario;
    -fx-border-radius: 4;
}

/* ===== Selector de ID: #fx:id ===== */
#mensajeLabel {
    -fx-font-style: italic;
}

/* ===== Clases que activamos o quitamos desde Java ===== */
.campo-error {
    -fx-border-color: -color-error;
    -fx-border-width: 2;
    -fx-border-radius: 4;
}

.mensaje-error {
    -fx-text-fill: -color-error;
}

.mensaje-ok {
    -fx-text-fill: -color-ok;
}
```

Archivo: `src/main/resources/com/duoc/javafxfxml/estilos-oscuro.css`

```
/* Tema oscuro: SOLO redefine las variables. Se carga DESPUÉS de estilos.css para que gane.
*/
.root {
    -color-primario: #4fc3f7;
    -color-fondo: #1e1e1e;
    -color-texto: #eeeeee;
    -color-error: #ef5350;
    -color-ok: #66bb6a;

    -fx-background-color: -color-fondo;
}

.label {
    -fx-text-fill: -color-texto;
}

.tarjeta {
    -fx-background-color: #2b2b2b;
}

.boton-primario {
    -fx-text-fill: #111111;
}
```

Archivo: `src/main/resources/com/duoc/javafxfxml/estilos-view.fxml`

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.control.ToggleButton?>
<?import javafx.scene.layout.VBox?>

<!-- stylesheets="@estilos.css": el @ significa "ruta relativa a este archivo" -->
<VBox fx:id="raiz" prefWidth="380.0" spacing="14.0" stylesheets="@estilos.css"
xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.duoc.javafxfxml.EstilosController">
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>
   <children>
      <Label styleClass="titulo" text="Laboratorio de estilos" />

      <VBox spacing="10.0" styleClass="tarjeta">
         <children>
            <Label text="Nombre" />
            <TextField fx:id="nombreField" promptText="Escribe tu nombre" />
            <Button onAction="#onValidarClick" styleClass="boton-primario" text="Validar"
/>
            <Label fx:id="mensajeLabel" />
         </children>
      </VBox>

      <Button disable="true" styleClass="boton-primario" text="Botón deshabilitado" />
```

```
      <ToggleButton fx:id="temaToggle" onAction="#onTemaClick" text="Tema oscuro" />
   </children>
</VBox>
```

Archivo: `src/main/java/com/duoc/javafxfxml/EstilosController.java`

```
package com.duoc.javafxfxml;

import javafx.fxml.FXML;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.control.ToggleButton;
import javafx.scene.layout.VBox;

public class EstilosController {

    @FXML private VBox raiz;
    @FXML private TextField nombreField;
    @FXML private Label mensajeLabel;
    @FXML private ToggleButton temaToggle;

    @FXML
    private void onValidarClick() {
        // Primero quitamos los estilos anteriores para no acumularlos
        nombreField.getStyleClass().remove("campo-error");
        mensajeLabel.getStyleClass().removeAll("mensaje-error", "mensaje-ok");

        if (nombreField.getText().isBlank()) {
            nombreField.getStyleClass().add("campo-error");
            mensajeLabel.getStyleClass().add("mensaje-error");
            mensajeLabel.setText("El nombre es obligatorio.");
        } else {
            mensajeLabel.getStyleClass().add("mensaje-ok");
            mensajeLabel.setText("¡Hola, " + nombreField.getText().trim() + "!");
        }
    }

    @FXML
    private void onTemaClick() {
        String oscuro = getClass().getResource("estilos-oscuro.css").toExternalForm();
        if (temaToggle.isSelected()) {
            raiz.getStylesheets().add(oscuro);      // se carga después => sus variables
ganan
        } else {
            raiz.getStylesheets().remove(oscuro);   // vuelve el tema claro
        }
    }
}
```

Ejecútalo:

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="estilos-view.fxml"
```

Lo importante de este ejemplo

1. **`stylesheets="@estilos.css"`** en el `VBox` raíz carga la hoja. El `@` significa “ruta **relativa a este FXML**”.

2. `styleClass="boton-primario"` le pone la clase al componente. Un mismo componente puede tener varias clases.
3. **Pseudo-clases:** `.boton-primario:hover` cambia el color cuando el mouse está encima; no necesitas ningún evento en Java.
4. **Clases dinámicas:** `getStyleClass().add("campo-error")` agrega un estilo desde Java; `remove(...)` lo quita. **Se quita antes de agregar** para no acumular clases.
5. **Tema oscuro:** `estilos-oscuro.css` **solo redefine las variables** de `.root` (más unos pocos ajustes). Se agrega **después**, por eso sus valores ganan (*en CSS, lo último definido pisa lo anterior*).
6. `toExternalForm()`**:** convierte la ruta del recurso en el texto que espera `getStylesheets()`.

Pruébalo

1. Ejecuta y presiona **Validar** con el campo vacío: aparece un borde rojo y un mensaje. Escribe un nombre y valida otra vez: ¿qué pasa con el borde rojo?
2. Pasa el mouse sobre el botón y mantenlo presionado. Mira cómo cambia el color (`:hover` y `:pressed`).
3. Presiona **Tema oscuro**. Luego **desactívalo**.
4. Cambia `-color-primario` en `estilos.css` a `#6a1b9a`. ¿Qué elementos cambian de color?
5. Haz clic dentro del campo de texto: ¿qué estilo se aplica? (`.text-field:focused`).
6. Quita la línea `raiz.getStylesheets().add(oscuro)` y reemplázala por `raiz.setStyle("-fx-background-color: black;")`. ¿Qué diferencia notas con el tema oscuro real? ¿Por qué es mejor la hoja de estilo?
7. Agrega la clase `.boton-peligro` (rojo) en `estilos.css` y un botón “Eliminar” que la use.
8. Cambia el orden: carga primero el oscuro y después el claro (en el controlador usa `raiz.getStylesheets().add(0, oscuro)`, que lo inserta **al principio**). Activa el tema oscuro. ¿Qué pasa y por qué?
9. Prueba el **visor con CSS externo** (segundo argumento):

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.VisorFxml -Djavafx.args="layouts-view.fxml estilos-oscuro.css"
```

Fíjate cómo el CSS se aplica a un FXML que **no sabe nada** de estilos. (Los dos nombres van **separados por un espacio** dentro de las mismas comillas.)

Comprueba lo que aprendiste

> ▼ **¿Qué diferencia hay entre `.boton` y `#boton`?**
>
> `.boton` es un selector de **clase** (puede aplicarse a muchos componentes con `styleClass="boton"`). `#boton` es un selector de **id** (solo el componente con `fx:id="boton"`).

▼ **¿Por qué el tema oscuro solo redefine variables?**

Porque todo el CSS base usa esas variables (`-color-primario`, `-color-fondo`…). Al **cambiar sus valores**, todo lo que las usa cambia automáticamente, sin repetir cada regla.

▼ **¿Por qué se quita una clase antes de agregar otra en `onValidarClick`?**

Para no **acumular** estilos contradictorios (por ejemplo, un campo con borde de error y de éxito a la vez) ni duplicar la misma clase al validar varias veces.

▼ **Si defino la misma propiedad en dos hojas de estilo, ¿cuál gana?**

La que se carga **después** (y, a igual orden, la de selector más específico: id > clase > tipo). El estilo en línea (`setStyle`) gana sobre las hojas.

★ Reto de la Etapa 3 · Tu propio tema

Toma el `layouts-view.fxml` o tu tarjeta de producto de la etapa 1 y crea `mi-tema.css` con:

1. Un esquema de **5 variables de color** en `.root` (primario, fondo, texto, énfasis, error).
2. Estilo para **al menos 3 tipos de control** (por ejemplo `.button`, `.label`, `.text-field`) usando selectores de **tipo**.
3. **Dos clases propias** (por ejemplo `.titulo` y `.tarjeta`) aplicadas con `styleClass`.
4. **Estados:** `:hover` y `:disabled` en los botones, `:focused` en los campos.
5. Un **tema alternativo** en otro `.css` que solo redefine variables, y un `ToggleButton` que alterne entre ambos.

Autoevaluación: ☐ no hay colores repetidos “a mano” (todo usa variables) · ☐ el FXML no tiene `style="..."` · ☐ cambiar el tema **no** requiere tocar el FXML · ☐ los textos se leen bien en ambos temas.

✎ **Foro:** publica **una captura de cada tema** y la regla CSS que más te sorprendió.

---

6. Etapa 4 · Reto integrador: formulario de registro

Une las tres etapas en una sola pantalla. Este reto **no incluye solución**: es para que lo resuelvas por tu cuenta y lo comentes en el foro.

Enunciado

Crea una aplicación **“Registro de estudiantes”** con `registro-view.fxml`, `RegistroController`, `registro.css` y la clase modelo `Estudiante`.

Componentes (etapa 1): - `BorderPane` como raíz, con `MenuBar` arriba (*Archivo → Limpiar*, *Archivo → Salir*). - Formulario en `GridPane`: **nombre**, **correo**, **carrera** (`ComboBox`), **año de ingreso** (`Spinner`), **género** (`RadioButton` en `ToggleGroup`), **fecha de nacimiento** (`DatePicker`), **acepta términos** (`CheckBox`). - Botones **Registrar** y **Limpiar**. - Una

`TableView<Estudiante>` con los registrados (nombre, correo, carrera). - Un `Label` de mensajes y otro con el total de registrados.

Eventos (etapa 2): - **Registrar** agrega el estudiante a la tabla (con validaciones, abajo). **Enter** en el campo de correo también registra. - **Limpiar** (botón y menú) deja todo vacío. - Seleccionar una fila de la tabla **carga sus datos** en el formulario. - Un botón **Eliminar** (deshabilitado si no hay fila seleccionada) con confirmación (`Alert`). - **Esc** limpia el formulario. - El botón **Registrar** está deshabilitado mientras **no** se marque "acepta términos". - El total de registrados se actualiza solo.

Validaciones: - Nombre y correo obligatorios; el correo debe contener `@` y no repetirse. - Debe elegirse carrera y género. - La edad calculada desde la fecha de nacimiento debe ser **al menos 16 años** (investiga `Period.between` o `ChronoUnit.YEARS`).

Estilos (etapa 3): - Todo el estilo en `registro.css` con **variables de color**. - Los campos inválidos reciben la clase `.campo-error` **y la pierden** cuando el usuario corrige el dato. - Mensajes de éxito y error con clases distintas. - Estado `:hover` / `:disabled` en los botones. - Un `ToggleButton` para cambiar entre **tema claro y oscuro**.

Diseño (POO): - `Estudiante` es una clase propia con atributos privados, constructor, *getters*, `equals` / `hashCode` por correo y `toString`. - La vista no contiene lógica; el controlador no construye la interfaz.

Rúbrica de autoevaluación

Criterio	Cumple	Parcial	No cumple
Usa al menos 6 tipos de control distintos	☐	☐	☐
Los eventos funcionan con mouse y teclado	☐	☐	☐
Los botones se habilitan y deshabilitan según el estado	☐	☐	☐
Las validaciones muestran mensajes claros y no hay caídas	☐	☐	☐
El estilo está 100 % en CSS (sin `style="..."`, salvo justificación)	☐	☐	☐
El tema oscuro se activa y desactiva sin errores	☐	☐	☐
`Estudiante` respeta encapsulamiento y `equals` / `hashCode`	☐	☐	☐
Puedes **explicar** cada parte del código a otra persona	☐	☐	☐

> ✎ **Foro:** publica tu proyecto (captura + 3 líneas: *qué hiciste · qué te costó · qué harías distinto*). Revisa el de **un compañero** y deja un comentario concreto: **una cosa que está bien y una que mejorarías**.

Anexo A · Resumen de componentes

Componente	Paquete	Uso	Métodos / propiedades clave
`Label`	`javafx.scene.control`	Mostrar texto	`setText`, `getText`, `setWrapText`
`Button`	idem	Ejecutar una acción	`setOnAction`, `setDisable`, `setDefaultButton`
`TextField`	idem	Texto de una línea	`getText`, `setText`, `clear`, `setPromptText`
`PasswordField`	idem	Clave	`getText`
`TextArea`	idem	Texto largo	`getText`, `setWrapText`, `appendText`
`CheckBox`	idem	Sí/No	`isSelected`, `setSelected`
`RadioButton`	idem	Una opción de varias	`setToggleGroup`, `isSelected`
`ToggleButton`	idem	Botón que queda presionado	`isSelected`, `setSelected`
`ComboBox<T>`	idem	Lista desplegable	`getValue`, `setValue`, `getItems`
`ListView<T>`	idem	Lista	`getItems`, `getSelectionModel()`
`TableView<T>`	idem	Tabla	`setItems`, `getColumns`, `getSelectionModel()`
`Spinner<T>`	idem	Número con flechas	`getValue`, `getValueFactory()`
`Slider`	idem	Valor en un rango	`getValue`, `setValue`, `valueProperty()`
`DatePicker`	idem	Fecha	`getValue` (`LocalDate`), `setValue`
`ProgressBar`	idem	Avance	`setProgress`, `progressProperty()`
`Alert`	idem	Diálogos	`setTitle`, `setHeaderText`, `showAndWait`
`MenuBar` / `Menu` / `MenuItem`	idem	Menús	`setOnAction`
`TabPane` / `Tab`	idem	Pestañas	`getTabs`, `getSelectionModel()`
`SplitPane`	idem	Paneles divisibles	`getItems`, `setDividerPositions`
`VBox` / `HBox`	`javafx.scene.layout`	Columna / fila	`setSpacing`, `setAlignment`, `getChildren`

Componente	Paquete	Uso	Métodos / propiedades clave
`BorderPane`	idem	5 zonas	`setTop/Left/Center/Right/Bottom`
`GridPane`	idem	Grilla	`add(nodo, col, fila)`, `setHgap`, `setVgap`
`StackPane`	idem	Apilado	`setAlignment`
`Region`	idem	Espacio flexible	`HBox.hgrow` / `VBox.vgrow`

Atributos de FXML que debes saber:

Atributo	Significado
`fx:id="nombre"`	Nombre para usar el componente en el controlador (`@FXML`)
`fx:controller="paquete.Clase"`	Controlador de la vista (en la raíz)
`onAction="#metodo"`	Método que atiende el evento (`#` obligatorio)
`styleClass="clase"`	Clase CSS
`stylesheets="@archivo.css"`	Hoja de estilos (ruta relativa al FXML)
`toggleGroup="$grupo"`	Usa un objeto definido en `<fx:define>`
`GridPane.columnIndex` / `rowIndex`	Celda dentro de un `GridPane`
`HBox.hgrow="ALWAYS"`	El hijo se estira para ocupar el espacio libre
`prefWidth` / `prefHeight` / `maxWidth="Infinity"`	Tamaños

Anexo B · Resumen de eventos

Evento	Atributo FXML	Parámetro del método	Datos útiles
Acción (botón, Enter, check, combo)	`onAction`	`ActionEvent`	`getSource()`
Clic de mouse	`onMouseClicked`	`MouseEvent`	`getButton()`, `getClickCount()`, `getX()`, `getY()`
Mouse presionado / soltado	`onMousePressed` / `onMouseReleased`	`MouseEvent`	idem
Mouse entra / sale	`onMouseEntered` / `onMouseExited`	`MouseEvent`	`getSource()`
Mouse se mueve / arrastra	`onMouseMoved` / `onMouseDragged`	`MouseEvent`	`getX()`, `getY()`
Tecla presionada / soltada	`onKeyPressed` / `onKeyReleased`	`KeyEvent`	`getCode()`, `isShiftDown()`, `isControlDown()`
Tecla escrita	`onKeyTyped`	`KeyEvent`	`getCharacter()`
Cambio de valor (código)	—	`(observable, anterior, nuevo)`	`propiedad.addListener(...)`

Tres reglas para no perderse: 1. El método de evento **puede tener 0 o 1 parámetro** (el del evento). 2. Lleva `@FXML` si es `private`. 3. El nombre del FXML (`#onGuardarClick`) debe ser **idéntico** al del método (sin `#`).

Anexo C · Resumen de CSS

Categoría	Propiedades
Color	`-fx-background-color`, `-fx-text-fill`, `-fx-opacity`
Texto	`-fx-font-size`, `-fx-font-weight`, `-fx-font-style`, `-fx-font-family`, `-fx-alignment`
Espacio	`-fx-padding`, `-fx-spacing` (VBox/HBox), `-fx-hgap`, `-fx-vgap` (GridPane)
Bordes y esquinas	`-fx-border-color`, `-fx-border-width`, `-fx-border-radius`, `-fx-background-radius`
Tamaño	`-fx-pref-width`, `-fx-min-width`, `-fx-max-width` (y las de alto)
Efectos	`-fx-effect: dropshadow(...)`
Ratón	`-fx-cursor: hand`

Valores de color: `red`, `#1f4e79`, `rgb(31,78,121)`, `rgba(0,0,0,0.5)`, `derive(color, 20%)`, `-mi-variable`.

Plantilla para una hoja de estilos ordenada:

```
/* 1. Variables */
.root { -color-primario: #1f4e79; }

/* 2. Estilos base por tipo */
.button { ... }
.label { ... }

/* 3. Clases propias */
.tarjeta { ... }

/* 4. Estados */
.button:hover { ... }

/* 5. Clases dinámicas (las activa el código Java) */
.campo-error { ... }
```

Referencia oficial: busca **“JavaFX CSS Reference Guide”** en `openjfx.io` (lista todas las propiedades `-fx-` y los nombres de estilo de cada control).

Anexo D · Errores frecuentes

Síntoma	Causa probable	Qué hacer
`Location is not set`	El nombre del `.fxml` está mal o no está en `resources/com/duoc/javafxfxml/`	Revisa nombre y carpeta; `mvn clean compile`
`Module ... does not open ... to javafx.fxml`	Falta el `opens` en `module-info.java`	`opens com.duoc.javafxfxml to javafx.fxml;`
`Error resolving onAction='#xxx'`	El método no existe o tiene otro nombre	Crea el método con `@FXML` y el mismo nombre
`NullPointerException` al usar un `@FXML`	El `fx:id` no coincide con el nombre del campo (distingue mayúsculas)	Igualarlos; no usarlo en el constructor, usar `initialize()`
`ClassNotFoundException` del controlador	`fx:controller` con paquete o nombre mal escrito	Copia el nombre completo de la clase
La columna de la tabla sale vacía	No existe el *getter* que busca `PropertyValueFactory`	Crear `getXxx()` con ese nombre
El CSS no se aplica	Ruta del `.css` incorrecta, error de sintaxis, o la clase no está asignada	Revisa el nombre; mira la consola: JavaFX muestra advertencias de CSS
Propiedad CSS "ignorada"	Usaste una propiedad web (`color`, `background-color`)	Usa las `-fx-…` (`-fx-text-fill`, `-fx-background-color`)
La clase CSS se acumula	Agregas la clase en cada clic sin quitarla	`remove(...)` antes de `add(...)`
Un estilo "no gana"	Otra regla lo pisa	Revisa el orden de carga y la especificidad (id > clase > tipo)
`A bound value cannot be set`	Haces `setText` / `setDisable` en una propiedad que tiene `bind`	Quita el `bind` o usa `unbind()`
`mvn: command not found`	Maven no está en el PATH	Reinstala o abre una terminal nueva

Cómo leer un error de Java: mira primero la línea `Caused by:` **más abajo**; suele decir el problema real ("Location is not set", "Can't convert…", etc.).

Anexo E · Glosario y recursos

Término	Significado
Nodo	Cualquier elemento del árbol visual
Layout	Contenedor que ordena a sus hijos
FXML	Lenguaje XML para describir la interfaz
Controlador	Clase Java que atiende los eventos de una vista
Evento	Algo que ocurre en la interfaz (clic, tecla, cambio)
Manejador (*handler*)	Método que atiende un evento
Listener	Observador que se ejecuta cuando una propiedad cambia
Propagación (*bubbling*)	Un evento sube del componente a sus padres
Binding	Enlace entre propiedades: una sigue a otra
CSS	Lenguaje para describir la apariencia
Selector	Parte del CSS que dice a qué componentes se aplica
Pseudo-clase	Estado de un componente (`:hover` , `:focused` …)
Hoja de estilos	Archivo `.css` con las reglas

Recursos: - Documentación de JavaFX: `openjfx.io` (API, CSS Reference Guide, tutoriales). - Scene Builder: `gluonhq.com/products/scene-builder`. - Tus guías: *JavaFX base con Maven* y *JavaFX con FXML y Scene Builder*.

Checklist final

Antes de la próxima semana deberías poder, **sin mirar los apuntes**:

✔	Puedo…
☐	Ejecutar cualquier `.fxml` con Maven y con IntelliJ.
☐	Elegir el layout adecuado (`VBox` , `HBox` , `BorderPane` , `GridPane` , `StackPane`).
☐	Usar `TextField` , `ComboBox` , `CheckBox` , `RadioButton` + `ToggleGroup` , `Spinner` , `DatePicker` y leer su valor.
☐	Llenar una `TableView` con objetos de una clase propia.
☐	Unir vista y controlador con `fx:controller` , `fx:id` y `onAction` .
☐	Responder a eventos de **acción**, **mouse** y **teclado**.
☐	Explicar la diferencia entre un manejador de evento y un listener.
☐	Escribir una hoja CSS con **variables**, **clases**, **ids** y **estados**.
☐	Aplicar y quitar clases CSS desde Java.
☐	Cambiar entre dos temas sin tocar el FXML.
☐	Leer un error de Java e identificar la causa.
☐	Resolver el reto integrador por mi cuenta.

Si todos están marcados, estás listo/a para la próxima semana. Si alguno no, **vuelve a esa etapa** y llévale la duda al foro.